
SOFTWARE REQUIREMENTS SPECIFICATION

for

GOLDEN EAGLE FLIGHT PLAN

Version 1.0

Prepared by: CS3338 Final Project Group

May 8, 2026

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Intended Audience and Reading Suggestions	4
1.3	Project Scope	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	User Classes and Characteristics	5
2.3	Product Functions	6
2.4	Operating Environment	6
2.5	Design and Implementation Constraints	7
2.6	Assumptions and Dependencies	7
3	System Features	8
3.1	Description and Priority	8
3.1.1	Start Objective Features	8
3.1.2	Checkpoint 1 Features	8
3.1.3	Checkpoint 2 Features	8
3.1.4	Final Checkpoint Features	9
3.2	Functional Requirements	9
3.2.1	Authentication Requirements	9
3.2.2	Navigation Requirements	9
3.2.3	Profile Requirements	9
3.2.4	Scoreboard Requirements	10
3.2.5	Resources and Events Feed Requirements	10
3.2.6	Faculty Dashboard Requirements	10
3.2.7	Personalization Engine Requirements	10
3.2.8	AI Feature Requirements	11
4	External Interface Requirements	12
4.1	User Interfaces	12
4.2	Hardware Interfaces	12
4.3	Software Interfaces	12
4.4	Communication Interfaces	12
5	Other Nonfunctional Requirements	13
5.1	Performance Requirements	13
5.2	Security Requirements	13
5.3	Software Quality Attributes	13
5.4	Business Rules	14
6	Database Requirements	15
6.1	Main Data Entities	15
6.2	Suggested Collections	15

7	Other Requirements	16
7.1	Future Enhancements	16
7.2	Maintenance Requirements	16
7.3	Appendix A: Glossary	16
7.4	Appendix B: References	16

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for the Golden Eagle Flight Plan application. The Golden Eagle Flight Plan is an AI-powered university information system designed to help students manage their academic, career, and leadership development in one centralized platform.

The system will help students organize university resources, track personal progress, view relevant events, and receive personalized recommendations. It will also allow faculty members to post resources and events while monitoring student engagement and progress.

This document describes the system's intended users, product scope, system features, functional requirements, interface requirements, nonfunctional requirements, and future enhancements.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for students, faculty members, developers, testers, project managers, and course staff.

Developers should focus on the system features, functional requirements, interface requirements, and database requirements. Testers should focus on the functional requirements, security requirements, performance requirements, and software quality attributes. Faculty members and course staff should focus on the project scope, user classes, scoreboard system, resources and events feed, and faculty dashboard requirements.

1.3 Project Scope

The Golden Eagle Flight Plan is a mobile application that centralizes student academic and professional development. The application will be built using React Native with Expo for the frontend, Node.js with Express for the backend, and MongoDB Atlas for cloud database storage.

The project will be developed in stages. The start objective focuses on building the foundation of the application, including the frontend structure, backend structure, database connectivity, user authentication, and a basic dashboard. Later checkpoints focus on implementing the user system, scoreboard system, resources and events feed, faculty dashboard, personalization engine, progress tracking, JWT security, performance optimization, scalability, and AI features.

The main goal is to help students stay organized and motivated by giving them a personalized platform for tracking progress and finding useful academic, career, and leadership opportunities.

2 Overall Description

2.1 Product Perspective

The Golden Eagle Flight Plan is a full-stack mobile application. It uses a frontend and backend architecture where the frontend handles the user interface and the backend handles application logic, database operations, authentication, and API requests.

The frontend will be built with React Native using Expo. It will include a navigation system with tabs such as Home, Scoreboard, Resources, AI, and Settings. These tabs allow users to quickly access the main parts of the application.

The backend will be built with Node.js and Express. It will handle API requests, user authentication, user roles, resources, events, progress tracking, and future AI-related requests. MongoDB Atlas will be used as the cloud database for storing users, resources, events, and progress data.

2.2 User Classes and Characteristics

The system has the following main user classes:

- **Student**

- Registers and logs into the application.
- Manages their personal profile.
- Views a personalized dashboard.
- Tracks progress through the scoreboard system.
- Views recommended resources and events.
- Uses future AI tools such as an academic advisor chatbot and resume feedback system.

- **Faculty Member**

- Registers or logs into the application with faculty access.
- Posts resources and events for students.
- Monitors student engagement.
- Reviews student progress through the faculty dashboard.

- **Administrator**

- Manages user access and system-level data.
- Maintains application settings.
- Helps manage database content and user roles.

2.3 Product Functions

The main functions of the Golden Eagle Flight Plan include:

1. **User Authentication**
The system allows users to register and log in securely.
2. **Role-Based Access**
The system separates student and faculty access so each user type has appropriate permissions.
3. **Dashboard Navigation**
The system provides a tabbed interface with Home, Scoreboard, Resources, AI, and Settings.
4. **Student Profile Management**
Students can manage profile information such as interests, major, and academic level.
5. **Scoreboard System**
Students can track achievements in Academic, Career, and Leadership categories.
6. **Resources and Events Feed**
Students can view resources and events related to their profile information.
7. **Faculty Dashboard**
Faculty members can post resources and events and monitor student engagement.
8. **Personalization Engine**
The system matches resources and events to users using hashtags based on profile information.
9. **Progress Tracking**
The system tracks completed tasks and calculates student levels and achievements.
10. **AI Features**
Future versions will include an academic advisor chatbot and resume feedback system.

2.4 Operating Environment

The system will operate in the following environment:

- Frontend framework: React Native with Expo.
- Backend framework: Node.js with Express.
- Database: MongoDB Atlas.
- Authentication: JWT authentication.
- Client devices: mobile devices, tablets, laptops, and desktop computers.
- Supported platforms: iOS, Android, and modern web browsers depending on deployment.

2.5 Design and Implementation Constraints

The system shall use React Native with Expo for the mobile frontend. The user interface shall use reusable components and a tabbed navigation system.

The backend shall use Node.js with Express to handle API requests and application logic. MongoDB Atlas shall be used to store application data. The system shall use JWT authentication for secure user session validation.

The project team will be divided into frontend and backend groups. The frontend group will focus on the user interface and navigation, while the backend group will focus on data management, API development, authentication, and database connectivity.

2.6 Assumptions and Dependencies

The system assumes that users will have internet access. The system also assumes that students will provide accurate profile information so that personalization can work correctly.

The application depends on Expo, Node.js, Express, MongoDB Atlas, and JWT authentication. If MongoDB Atlas or the backend server is unavailable, users may not be able to log in, update progress, or view current resources and events.

3 System Features

3.1 Description and Priority

The system features are prioritized according to the project checkpoints.

3.1.1 Start Objective Features

1. Establish frontend application structure using React Native with Expo.
2. Establish backend application structure using Node.js with Express.
3. Connect the backend to MongoDB Atlas.
4. Create login and registration screens.
5. Create a basic dashboard layout.
6. Create a tabbed navigation system with Home, Scoreboard, Resources, AI, and Settings.

3.1.2 Checkpoint 1 Features

1. Implement user registration and login.
2. Implement student and faculty roles.
3. Allow users to manage profiles.
4. Implement the scoreboard system.
5. Implement resources and events feed.
6. Implement the faculty dashboard.

3.1.3 Checkpoint 2 Features

1. Implement personalization using hashtags.
2. Match resources and events to student profile information.
3. Track completed tasks.
4. Calculate levels and achievements.
5. Implement JWT authentication.
6. Improve user interface and navigation.

3.1.4 Final Checkpoint Features

1. Optimize application performance.
2. Improve scalability.
3. Begin integrating AI features.
4. Add or prepare the academic advisor chatbot.
5. Add or prepare the resume feedback system.
6. Improve accessibility and usability for students and faculty.

3.2 Functional Requirements

3.2.1 Authentication Requirements

1. The system shall allow users to register an account.
2. The system shall allow users to log into an existing account.
3. The system shall validate login credentials.
4. The system shall assign users a role of student, faculty, or administrator.
5. The system shall use JWT authentication for user session validation.
6. The system shall prevent unauthenticated users from accessing protected features.
7. The system shall allow users to log out.

3.2.2 Navigation Requirements

1. The system shall provide a tabbed navigation interface.
2. The navigation system shall include Home, Scoreboard, Resources, AI, and Settings.
3. The system shall allow users to switch between tabs.
4. The system shall show user-specific content based on role.

3.2.3 Profile Requirements

1. The system shall allow students to create a profile.
2. The system shall allow users to update profile information.
3. The student profile shall include academic level.
4. The student profile shall include major.
5. The student profile shall include interests.
6. The system shall use profile information for personalization.

3.2.4 Scoreboard Requirements

1. The system shall provide a scoreboard for students.
2. The scoreboard shall track Academic category progress.
3. The scoreboard shall track Career category progress.
4. The scoreboard shall track Leadership category progress.
5. The system shall allow students to view completed and incomplete tasks.
6. The system shall store task completion information.
7. The system shall calculate levels and achievements based on completed tasks.

3.2.5 Resources and Events Feed Requirements

1. The system shall display resources to students.
2. The system shall display events to students.
3. The system shall recommend resources based on user interests.
4. The system shall recommend resources based on major.
5. The system shall recommend resources based on academic level.
6. The system shall recommend events based on user interests.
7. The system shall recommend events based on major.
8. The system shall recommend events based on academic level.

3.2.6 Faculty Dashboard Requirements

1. The system shall provide a dashboard for faculty members.
2. The system shall allow faculty members to post resources.
3. The system shall allow faculty members to post events.
4. The system shall allow faculty members to monitor student engagement.
5. The system shall allow faculty members to view student progress.

3.2.7 Personalization Engine Requirements

1. The system shall use hashtags to classify resources and events.
2. The system shall compare hashtags with student profile information.
3. The system shall recommend matching resources to students.
4. The system shall recommend matching events to students.
5. The system shall update recommendations when profile information changes.

3.2.8 AI Feature Requirements

1. The system should include an AI section in the navigation system.
2. The system should support future integration of an academic advisor chatbot.
3. The system should support future integration of a resume feedback system.
4. The system should allow AI features to use student profile and progress data only when authorized.

4 External Interface Requirements

4.1 User Interfaces

The system shall provide a mobile-friendly interface. The user interface shall include screens for registration, login, dashboard, scoreboard, resources, AI tools, settings, profile management, and faculty dashboard.

The dashboard should be easy to navigate and should clearly display important information. The scoreboard should show student progress in Academic, Career, and Leadership categories. The resources and events feed should show personalized opportunities based on the student profile.

4.2 Hardware Interfaces

The system does not require special hardware. Users may access the application using smartphones, tablets, laptops, or desktop computers.

4.3 Software Interfaces

The frontend shall communicate with the backend using API requests. The backend shall communicate with MongoDB Atlas for data storage and retrieval. JWT authentication shall be used to secure communication and validate sessions.

4.4 Communication Interfaces

The system shall communicate over standard network protocols. When deployed, the system should use HTTPS to protect user information and authentication tokens.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- The application should load main screens within a reasonable time.
- The dashboard should display updated information quickly.
- The resources and events feed should retrieve personalized results efficiently.
- Scoreboard updates should be saved and displayed without unnecessary delay.
- The system should remain responsive as more users and data are added.

5.2 Security Requirements

- The system shall use JWT authentication.
- The system shall protect user sessions.
- The system shall restrict access based on user role.
- Students shall not be able to access faculty-only features.
- Faculty members shall not be able to modify system settings unless given administrator access.
- Passwords shall be stored securely.
- The system should use HTTPS when deployed.

5.3 Software Quality Attributes

- **Usability:** The system should be easy for students and faculty to understand and navigate.
- **Accessibility:** The system should be designed to support users with different accessibility needs.
- **Reliability:** The system should accurately store user data, profile information, resources, events, and progress.
- **Maintainability:** The codebase should be organized so future features can be added easily.
- **Scalability:** The system should support more users, resources, events, and progress records over time.
- **Performance:** The system should be optimized for fast loading and smooth navigation.

5.4 Business Rules

- Each user must have a role.
- Students may only update their own profile and progress information.
- Faculty members may post resources and events.
- Faculty members may monitor student engagement.
- Resources and events should be categorized with hashtags.
- Personalized recommendations should be based on user interests, major, and academic level.
- Scoreboard achievements should be based on completed tasks.
- AI features should not access private student information unless authorized.

6 Database Requirements

6.1 Main Data Entities

The MongoDB Atlas database shall store data for the following entities:

- **Users:** Stores account information, login credentials, role, and profile data.
- **Profiles:** Stores student academic level, major, interests, and related personalization data.
- **Resources:** Stores resource title, description, category, hashtags, and creator information.
- **Events:** Stores event title, description, date, category, hashtags, and creator information.
- **Scoreboard Tasks:** Stores Academic, Career, and Leadership tasks.
- **Progress Records:** Stores completed tasks, levels, and achievements.

6.2 Suggested Collections

The database may include the following collections:

- users
- profiles
- resources
- events
- scoreboard_tasks
- progress_records

7 Other Requirements

7.1 Future Enhancements

Future versions of the Golden Eagle Flight Plan may include a fully integrated academic advisor chatbot, resume feedback system, stronger recommendation algorithms, notifications, advisor notes, progress reports, and additional university system integrations.

7.2 Maintenance Requirements

The system will require regular maintenance to fix bugs, update dependencies, improve security, optimize performance, and add new resources, events, and AI capabilities.

7.3 Appendix A: Glossary

- **Golden Eagle Flight Plan:** The university information system described in this SRS.
- **Scoreboard:** A progress tracking feature for academic, career, and leadership achievements.
- **Resource:** A useful academic, career, or leadership support item posted in the system.
- **Event:** A university activity or opportunity shown to users.
- **Faculty Dashboard:** A faculty-facing interface for posting resources and events and monitoring student engagement.
- **JWT:** JSON Web Token, used to validate secure user sessions.
- **Personalization Engine:** The part of the system that matches resources and events to users based on profile hashtags.

7.4 Appendix B: References